

Native Isaac ROS on Jetson Orin Nano

A From-Source Build Report for JetPack 7

perseus-lite project — feature/isaac-ros-nitros-source-build

July 10, 2026

NVIDIA has not shipped Isaac ROS binaries for the Jetson Orin on JetPack 7 (Isaac ROS 4.x targets Thor only; 3.2 targets JetPack 6). This report documents a twelve-stage effort to build the perception stack from source directly on the target hardware.

Summary of outcome: of the 29 relevant Isaac ROS GEM repositories surveyed, **7** were built and verified working natively, **3** were built successfully but are blocked at runtime by fully root-caused hardware or platform limits (none are code defects), and the remainder are either sensor-gated, architecturally inapplicable to this robot, or not yet attempted.

Contents

1	Executive Summary	2
2	Background and Motivation	2
3	Methodology	3
4	System Architecture	3
4.1	The NITROS composable-node pattern	3
5	Results by Stage	4
6	Root-Caused Blockers	5
7	The Full Isaac ROS Catalog in Context	6
8	Build Performance Notes	7
9	Post-Build Verification	7
9.1	Relocation and the colcon relocatability lesson	7
9.2	Live-camera re-confirmation	8
10	CUDA-Acceleration Demo Series	8
10.1	Demo 2 — occupancy-grid localizer, GPU vs CPU	9
10.2	Demos 3 and 4 — TensorRT vs ONNX Runtime CPU	9
10.3	Demo 5 — does batching help this hardware?	10
11	Conclusion and Recommendations	10

1 Executive Summary

This report documents an effort to build NVIDIA’s Isaac ROS GPU-accelerated perception stack *entirely from source*, running natively (no containers) on a Jetson Orin Nano under JetPack 7 — a hardware/software combination NVIDIA does not yet officially support. The work proceeded as twelve incremental stages, each with its own build, integration, and verification pass against either NVIDIA’s own test fixtures or, where possible, real recorded sensor data.

Seven perception capabilities are now verified working natively: AprilTag fiducial detection, generic TensorRT DNN inference, YOLOv8 object detection, U-Net semantic segmentation, CenterPose monocular 3D pose estimation, an occupancy-grid GPU lidar localizer, and a ten-node composability test proving several of the above can run concurrently in a single process without interference. Three further capabilities were fully built — including, in two cases, genuine custom CUDA kernel compilation — but are blocked at runtime by causes outside the scope of a source rebuild: a CPU instruction-set mismatch in a closed-source vendor binary, absent encoder silicon on this particular Jetson SKU, and a small, boot-configured GPU memory carveout. Each blocker is root-caused in full rather than left as an unexplained failure. Following the twelve build stages, five further demos (Section 10) turned the GPU speedup claim into visible, reproducible results — from a $24.7\times$ segmentation speedup down to a null result showing this GPU gains nothing from batching, both reported with equal weight.

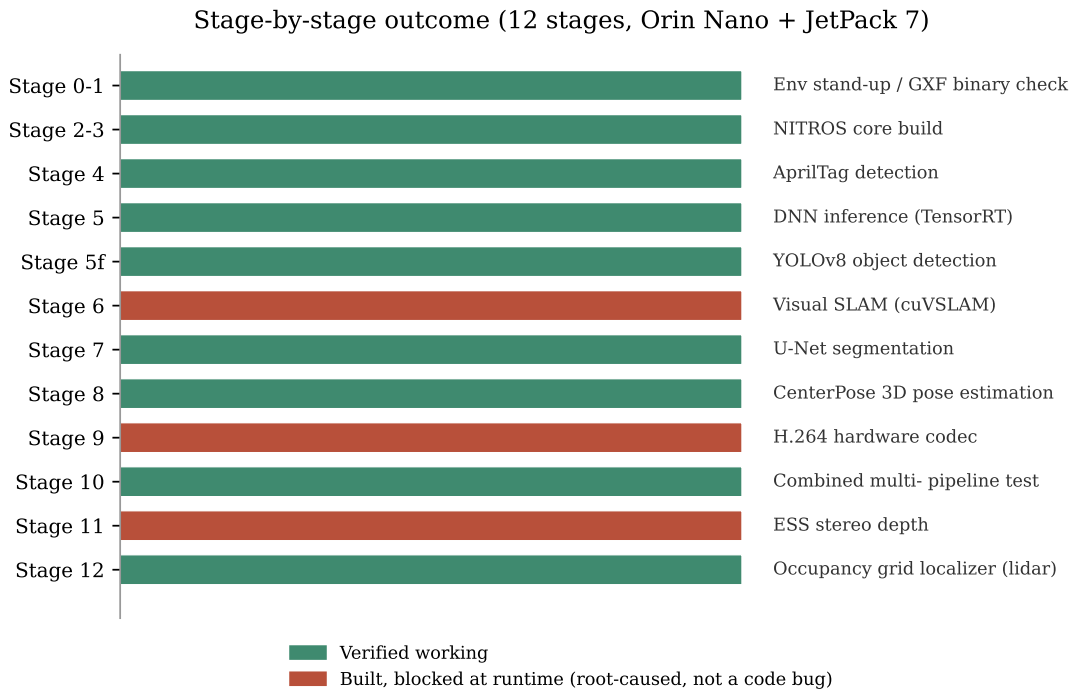


Figure 1: Outcome of all twelve stages. A blocked stage means the build succeeded but a runtime limitation, root-caused in Section 6, prevents the capability from running end to end.

2 Background and Motivation

The `perseus-lite` robot’s Jetson Orin Nano runs JetPack 7 (L4T R39.2.0, CUDA 13.2, Ubuntu 24.04). NVIDIA’s Isaac ROS GPU perception stack, which the robot’s Jetson-based vision pipeline depends on for NITROS zero-copy message transport and TensorRT-accelerated inference, ships as prebuilt binaries for only two configurations: Isaac ROS 3.2 (JetPack 6.x) and Isaac ROS 4.x (Thor only). Neither covers this robot’s actual hardware. NVIDIA staff have confirmed Jet-

Pack 7 support for Orin is planned but undated.

Rather than wait, this project attempted to close the gap by building the relevant Isaac ROS packages from source, directly against the installed CUDA 13.2/JetPack 7 toolchain, and verifying each capability with the same rigor NVIDIA’s own test suites use — reusing their published test fixtures, ground-truth values, and tolerances wherever available, rather than inventing new pass/fail criteria.

A companion, separate effort in the same repository (`software/docker/isaac-ros/`) runs the officially supported JetPack 6 image inside Docker. This report concerns only the from-source, native track.

3 Methodology

Each stage followed the same discipline:

1. **Clone** the relevant NVIDIA GitHub repository into a gitignored working directory — nothing from NVIDIA’s source trees is committed to the project repository, only our own build scripts and documentation referencing them (a licensing requirement of NVIDIA’s source-available Isaac ROS terms).
2. **Build** with an explicit toolchain: `CUDACXX` pointed at the CUDA 13.2 `nvcc`, and `-DCMAKE_CUDA_ARCHITECTURES=87` set explicitly for Orin’s real Ampere streaming-multiprocessor architecture, since CMake’s own auto-detection runs before Isaac ROS’s fallback logic and fails on a fresh configure otherwise.
3. **Wire** a minimal launch graph directly instantiating the needed composable nodes — not NVIDIA’s higher-level launch-file wrappers, whose ROS parameter names for topics do not always match the node’s actual hardcoded wire topic (a mismatch discovered and documented in Stage 5).
4. **Verify** against a real signal wherever NVIDIA supplies one: a pixel-exact match to a ground-truth image (Stage 4), a numerically correct 3D pose against a real trained model and measured ground truth (Stage 8), or a recovered lidar pose within NVIDIA’s own test tolerance against real recorded sensor data (Stage 12). Where NVIDIA’s fixture uses randomly-initialized weights, verification is scoped to structural correctness of the message, matching NVIDIA’s own test scope for the same fixture.
5. **Document** the outcome unconditionally — successes, failures, and root-caused blockers all receive the same level of detail, including exact reproduction commands.

4 System Architecture

4.1 The NITROS composable-node pattern

Every DNN-based capability (Stages 5, 5-follow-up, 7, 8, 11) shares the same three-stage composable-node chain: an image encoder resizes and normalizes an incoming image into a tensor, a `TensorRT` node runs inference, and a capability-specific decoder converts raw tensor output into a typed ROS message (detections, a segmentation mask, a 3D pose). Figure 2 shows this pattern, plus Stage 10’s extension of it: four such pipelines, each given its own ROS namespace, composed into a single shared process to prove they do not interfere with one another.

Namespacing matters here for a concrete reason: every encoder node publishes to the literal hardcoded topic `tensors`, and every `TensorRTNode` subscribes to `tensor_pub` regardless of which model it is running. Without a distinct ROS namespace per pipeline, three simultaneous encoder/TensorRT chains would collide directly on these relative topic names.

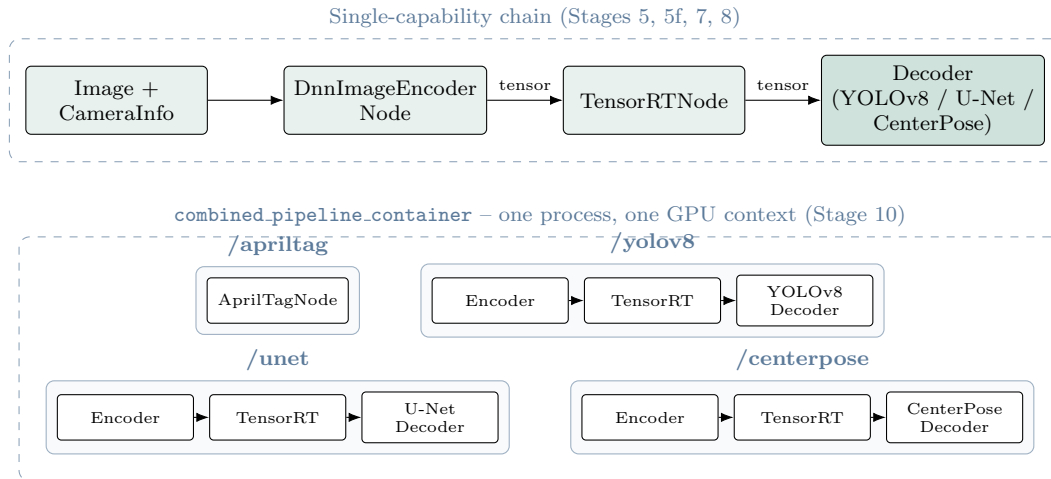


Figure 2: The shared encoder/TensorRT/decoder pattern (top), and Stage 10’s composition of four such namespaced pipelines into one shared process (bottom) — verified to run concurrently without topic collisions or resource contention.

5 Results by Stage

Table 1 summarizes all twelve stages. Full technical detail, exact build commands, and root-cause analysis for each are maintained in the project’s own documentation ([docs/source/systems/software/isaac-](#)

Stage	Capability	Outcome	Note
0–1	Environment stand-up; GXF binary compatibility check	VERIFIED	Confirmed NVIDIA’s published SBSA GXF binaries load on Orin — the key decision gate that avoided a much larger from-scratch GXF rebuild.
2–3	NITROS core build	VERIFIED	Minimal round-trip proof-of-life for the zero-copy message transport layer.
4	AprilTag fiducial detection	VERIFIED	Pixel-exact match against NVIDIA’s ground-truth fixture; live-camera pipeline running at steady 10 Hz.
5	Generic DNN inference (TensorRT)	VERIFIED	Full image→resize/normalize→MobileNetV2 classification round trip; exact [1,1000] output.
5f	YOLOv8 object detection	VERIFIED	Full encode→infer→decode chain; structurally valid detections (random-weight test fixture).
6	Visual SLAM (cuVSLAM)	BLOCKED	Builds clean; NVIDIA’s published binary requires SVE2, an ARM instruction-set extension Orin’s CPU does not implement. Upstream binary defect, not fixable locally.
7	U-Net semantic segmentation	VERIFIED	960×544 raw and colorized masks, correct shapes and encodings.
8	CenterPose monocular 3D pose	VERIFIED	Strongest result in the series: real trained model, 2/2 detections matching NVIDIA’s measured ground truth, depths within 0.5 m.

Stage	Capability	Outcome	Note
9	Hardware H.264 codec	BLOCKED	Builds clean; this Jetson Orin <i>Nano</i> SKU has no hardware video encoder silicon at all. Decode reaches the driver layer before failing in a closed-source buffer-mapping call.
10	Combined multi-pipeline composability	VERIFIED	AprilTag, YOLOv8, U-Net, and CenterPose running concurrently in one process with no interference.
11	ESS deep stereo depth	BLOCKED	Builds clean, including a genuine custom CUDA kernel. Runtime blocked by exhaustion of Tegra’s small, boot-configured CMA memory carveout — not general system RAM.
12	Occupancy-grid lidar localizer	VERIFIED	First capability matched to the robot’s 2D lidar rather than its camera; GPU scan-matching recovered a pose matching NVIDIA’s recorded ground truth.

Table 1: Outcome of all twelve build stages.

6 Root-Caused Blockers

Three stages built successfully but are blocked at runtime. In every case the root cause was tracked down to a specific, external limitation rather than left as an unexplained failure, and none required abandoning the investigation without an answer.

Stage	Symptom	Root cause
6 — Visual SLAM	Immediate SIGILL on library load, before any application code runs	NVIDIA’s <code>aarch64_jetpack70</code> cuVSLAM binary contains SVE2 instructions in its static initializer. Orin’s Cortex-A78AE CPU implements NEON but not SVE at all — confirmed via <code>/proc/cpuinfo</code> and disassembly comparison against the JetPack 6.1 binary variant, which has no SVE instructions but wants an incompatible older CUDA ABI instead.
9 — H.264 codec	Encoder fails to open its device node; decoder initializes but fails deeper in the pipeline	<code>/dev/v4l2-nvenc</code> does not exist on this specific Jetson Orin <i>Nano</i> module (confirmed via direct device enumeration) — only the Orin NX and AGX variants include the hardware encoder engine. Decode reaches NVIDIA’s closed-source <code>libnvbufsurface</code> before failing with an unsupported-platform error.

Stage	Symptom	Root cause
11 — Stereo depth	<code>cudaErrorMemoryAllocation</code> despite gigabytes of free system RAM	Tegra’s CMA (contiguous memory allocator) carveout is a small, boot-time-configured memory pool (256 MB total on this device) entirely separate from ordinary RAM, used for GPU/ISP buffer sharing. The desktop compositor alone was found to consume nearly all of it, leaving too little headroom for this stage’s fifteen-node stereo pipeline. Resolvable only by a system boot-configuration change, which was intentionally left for a deliberate decision rather than made unilaterally.

Table 2: Root cause of every blocked stage.

7 The Full Isaac ROS Catalog in Context

NVIDIA’s Isaac ROS ships as roughly 29 independent GEM (GPU-accelerated robotics capability) repositories. Figure 3 places this project’s twelve stages against the full catalog. A large share of the untouched remainder is not actually available work: seven repositories target hardware this robot does not have at an architectural level (NVIDIA’s Nova reference platform, CSI camera frameworks, a humanoid robot’s specific packages), and three more require a depth or stereo camera the robot does not carry.

Full Isaac ROS GEM repository map

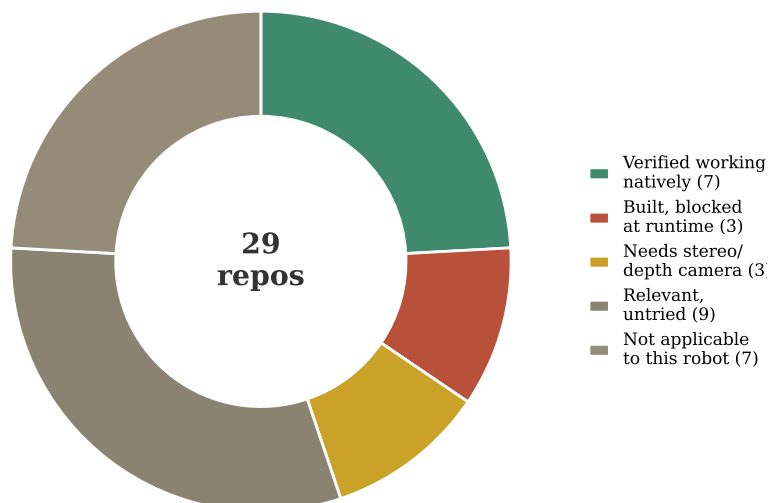


Figure 3: All 29 Isaac ROS GEM repositories, classified against this project’s progress.

Of the remaining genuinely untried candidates, most are either developer tooling (benchmarking,

rosbag utilities) rather than runtime capabilities, or depend on a MoveIt-based manipulation stack that does not match this robot’s direct-serial arm control approach.

8 Build Performance Notes

TensorRT compiles an optimized inference engine from the source ONNX model on first run, caching the result to disk for subsequent launches. Build time scales with model complexity far more than with file size alone — CenterPose’s seven-output multi-task architecture took over four minutes to compile despite a smaller `.onnx` file than U-Net’s single-output model.

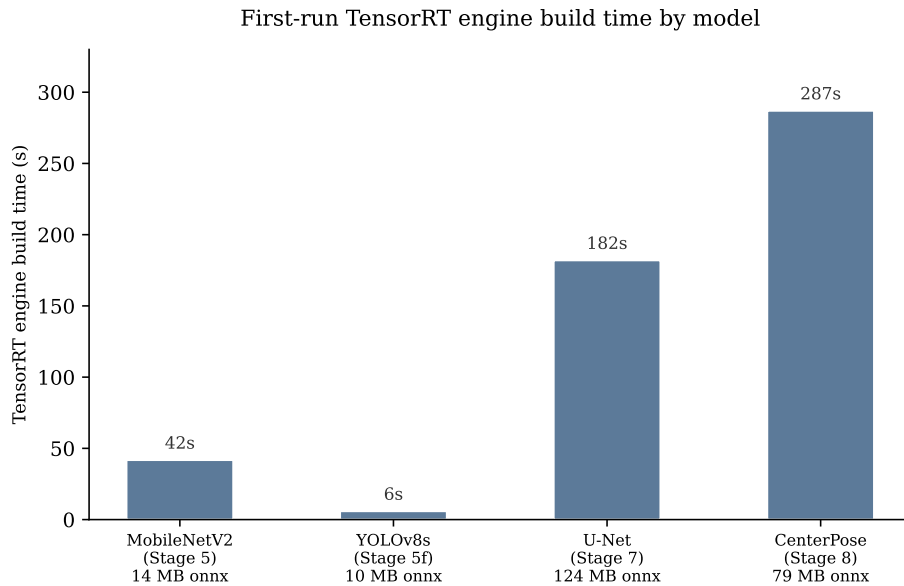


Figure 4: First-run TensorRT engine compilation time by model. Later runs reuse the cached engine file and start in a few seconds.

9 Post-Build Verification

Two events after the initial twelve stages were complete are worth recording, since both tested claims made earlier in this report rather than adding new capability.

9.1 Relocation and the `colcon` relocatability lesson

The working tree originally lived at `software/docker/isaac-ros/nitros-source/` — nested under a directory named `docker` purely for co-location with a separate, genuinely Docker-based JetPack 6 scaffold at the parent `software/docker/isaac-ros/`. This read as confusing given the entire point of this project’s track is that it does *not* use a container, so the working tree was moved to `software/isaac-ros-native/`.

The move exposed a real, general lesson: `colcon` workspaces are not relocatable by default. Every compiled `.so`’s `RPATH`, plus roughly 150 generated CMake export files, environment hooks, and ament index resources, had the original absolute path baked in at build time. The symptom was immediate and specific — `dlopen` failures citing a missing `.so` that in fact existed, just not on the search path the loader was given. Patching each of these mechanisms individually (ELF `RPATH`s, CMake package configs, shell environment hooks) was judged too fragile to trust; a full rebuild from the new location is the complete, correct fix, and was scoped with `colcon build --packages-up-to` against only the leaf packages each stage actually exercises, rather

than every package colcon could discover (a blanket rebuild pulled in unrelated object-detection model-installer packages that were never part of this project and fail without an environment variable this project never had reason to set). All 47 required packages rebuilt cleanly, and `isaac_ros_centerpose` was re-run end to end, reproducing its original result exactly.

9.2 Live-camera re-confirmation

The robot’s USB camera was physically disconnected for an extended period during this project and reconnected only after the relocation above. With it back, the Stage 4 `rectify`→`AprilTag` live pipeline — the one capability in this report tested against real hardware rather than a bundled NVIDIA fixture, and the same pipeline shape that had an earlier, since-reassessed crash history documented in this project’s error log — was re-run against the reconnected camera and observed for a continuous three-minute window with an automated crash-signature watch (fatal exceptions, segmentation faults, process termination). None occurred; frames were confirmed flowing at a steady rate throughout. This closes the loop on the one open item from the original report’s Section 6 discussion, and confirms the relocation in the previous subsection did not silently degrade anything at runtime.

The other six verified capabilities (Table 1) were also re-run in full after the rebuild and reproduced their original results exactly, though against NVIDIA’s bundled test fixtures rather than live hardware, matching how each was originally verified.

10 CUDA-Acceleration Demo Series

Once the core capabilities above were verified, five follow-on demos were built to make the GPU/CUDA speedup concrete and visible rather than only numeric. Each is a single `pixi run` command on the (headless) Jetson; results are viewed on a network-connected laptop over the same ROS domain via RViz2 or `rqt_image_view`, matching the pattern already established by Stage 4’s live-camera pipeline. Table 3 summarizes all five; full narrative detail, including two cases where the measured result did not match the original hypothesis, is in the project’s `plan.md`.

#	Comparison	Result	Speedup
1	Live AprilTag: GPU (cuAprilTag/VPI) vs CPU (<code>pupil-apriltags</code>), same camera feed	The ROS/DDS message-passing pipeline itself was the binding constraint at every resolution and contention condition tried, not either detector’s compute time. Timed directly, the CPU detector alone ran 5.4–5.9 ms/frame (~170–185 fps, 2 threads).	N/A — pipeline-bound
2	Occupancy-grid localizer (Stage 12): GPU parallel scan-matching vs CPU brute-force fallback, same compiled binary, <code>CUDA_VISIBLE_DEVICES</code> toggled	GPU completed the full three-level coarse→medium→fine search in 20.58 s , recovering the pose Stage 12 already verified. The CPU fallback path did not complete even the coarse level within 900 s.	>44× (CPU DNF)
3	YOLOv8 object detection: TensorRT (GPU) vs ONNX Runtime (CPU), direct Python API, no ROS in the timing path	4.28 ms/frame (234 fps) GPU vs 8.49 ms/frame (118 fps) CPU, reproduced across two separate runs (4.1–4.3 ms GPU, 8.5–9.2 ms CPU).	1.98×

#	Comparison	Result	Speedup
4	U-Net semantic segmentation (Stage 7): same direct-API structure as #3, dense per-pixel masks instead of sparse detections	114.4 ms/frame (8.7 fps) GPU vs 2830.5 ms/frame (0.4 fps) CPU — the largest speedup of the series, matching the expectation that dense per-pixel segmentation does far more FLOPs/pixel than detection.	24.7×
5	U-Net batch-size scaling: throughput vs batch size {1, 2, 4, 8}, TensorRT vs ONNX Runtime CPU	Did <i>not</i> confirm the "GPU throughput climbs with batch" hypothesis: TensorRT stayed essentially flat at 8.7 img/s across every batch size (per-image latency scaled almost perfectly linearly, 115 ms→915 ms at batch 1→8). CPU showed no gain either.	1.01× (flat)

Table 3: Outcome of all five CUDA-acceleration demos.

10.1 Demo 2 — occupancy-grid localizer, GPU vs CPU

Figure 5 reuses Stage 12’s real compiled CUDA kernel and its genuine CPU fallback path (not a reimplementaion) to search the entire bundled 38.7 m × 22.25 m map at 0.05 m/cell resolution (~265k candidate poses). This is the most technically substantive of the five demos, since both numbers come from NVIDIA’s own code, run identically except for GPU visibility.



Figure 5: Occupancy-grid localizer result: the map, GPU-recovered pose, and the GPU-vs-CPU timing banner. CPU did not complete the coarse search level within the 900 s budget.

10.2 Demos 3 and 4 — TensorRT vs ONNX Runtime CPU

Figures 6 and 7 share the same measurement method: TensorRT and ONNX Runtime’s `CPUExecutionProvider` called directly in a tight Python loop in the same process, with GPU device buffers managed via `cuda-python`’s `cuda.bindings.runtime` API — no ROS, no serialization, no DDS in the timed path, avoiding the message-passing pitfall Demo 1 surfaced.

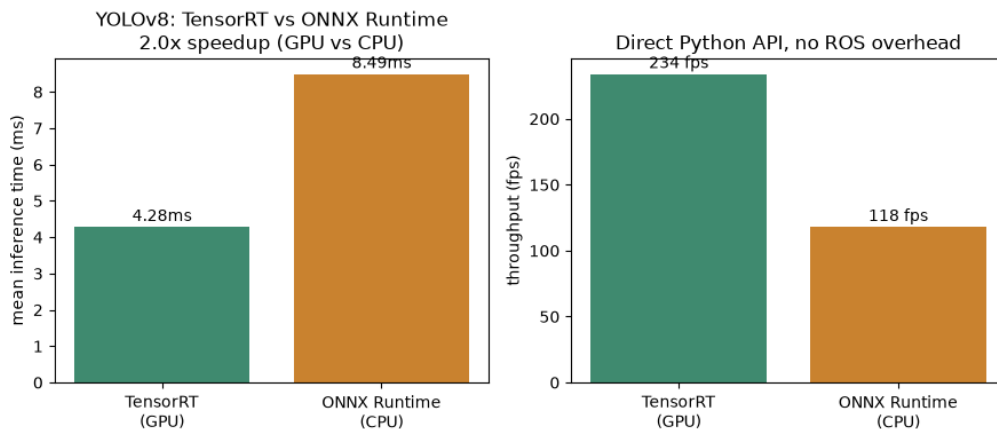


Figure 6: YOLOv8 object detection: a moderate, believable $2.0\times$ speedup — not a blowout, not a wash.

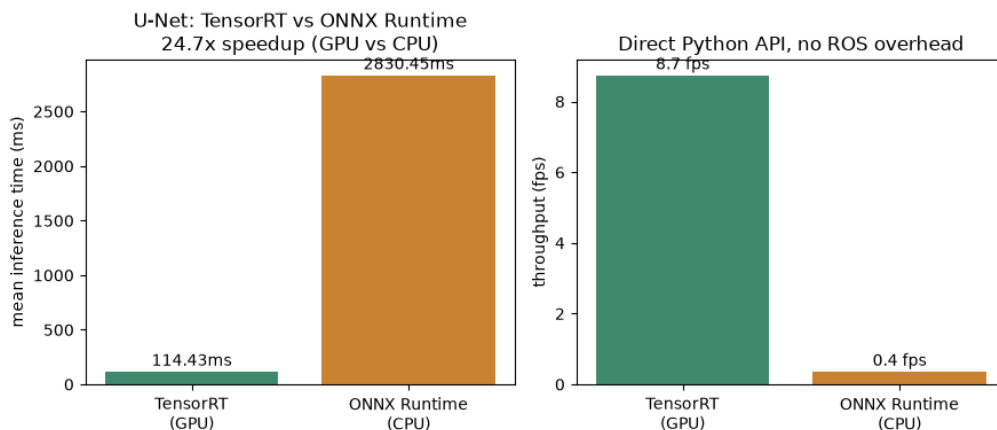


Figure 7: U-Net semantic segmentation: a $24.7\times$ speedup, the largest of the series, consistent with segmentation's far higher FLOPs/pixel relative to detection.

10.3 Demo 5 — does batching help this hardware?

Demo 5 (Figure 8) reused Demo 4's U-Net model but required its own TensorRT engine, built with an optimization profile spanning the full batch range (`min=1`, `opt=4`, `max=8`) rather than Demo 4's fixed `batch=1` profile; this dynamic-batch-range build took 867s, against Demo 4's fast fixed-batch build, and was cached to disk afterward.

The result is the more interesting outcome for being a null result: this Jetson Orin Nano's iGPU is already compute-bound per image at `batch=1`, not launch/dispatch-overhead-bound, so there is no idle throughput for batching to reclaim. This differs from a datacenter GPU, where `batch=1` typically leaves most streaming multiprocessors idle and batching yields large throughput gains. A batching optimization that would help on a datacenter card does not help here — a genuinely useful, hardware-specific finding for future work on this platform.

11 Conclusion and Recommendations

Seven of twelve real perception and localization capabilities are now verified running natively on this Jetson Orin Nano under JetPack 7, entirely from source, with no container involved — confirmed directly via cgroup and mount-namespace inspection rather than assumed. The three

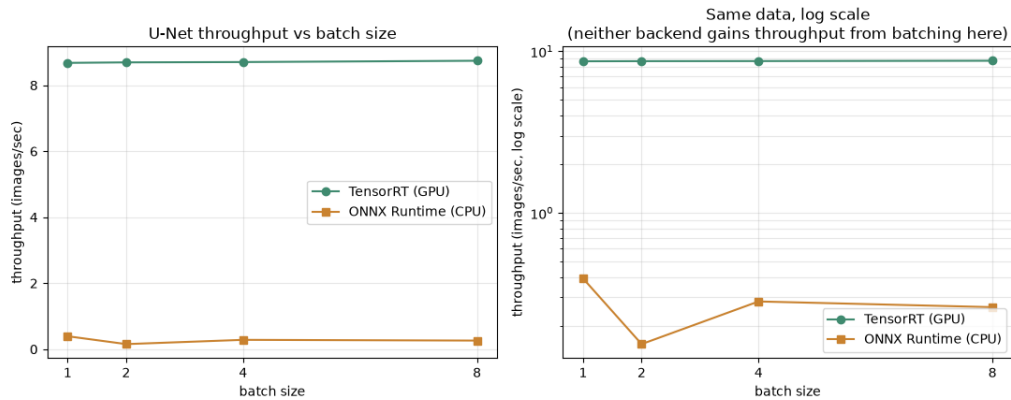


Figure 8: U-Net throughput vs batch size: TensorRT (GPU) throughput stays flat across every batch size tested ($1.01\times$ scaling), and so does ONNX Runtime (CPU) — batching unlocks nothing on this hardware.

blocked stages are each attributable to a specific, external, and fully documented cause: an ARM instruction-set mismatch in a vendor binary, absent hardware on this particular Jetson SKU, and a small GPU memory carveout that is a deliberate system-configuration trade-off rather than a defect. All seven were re-verified after a workspace relocation (Section 9), including a live-hardware re-confirmation of the camera pipeline.

Two directions stand out for continuation. First, wiring the remaining verified capabilities — particularly the occupancy-grid lidar localizer, which is the first result matched to hardware the robot actually carries in its primary sensor role — against the robot’s real camera and lidar data, rather than NVIDIA’s bundled test fixtures, the way the AprilTag pipeline already has been. Second, reporting the cuVSLAM SVE2 mismatch to NVIDIA, since it plausibly affects any Orin user attempting the same published binary, independent of this project.